

The Effects of Leadership in Distributed Consensus

An Experimental Comparison of Leader-Based and Leaderless Consensus

Sephy Ryu
Independent Researcher
South Korea

Abstract

Leader-based and leaderless consensus protocols differ in how replicas coordinate operations and recover from failures. This study experimentally examines these differences using Raft and EPaxos as representative leader-based and leaderless designs. We evaluate throughput, latency, and failure behaviour across replica count, client concurrency, read/write composition, conflict rate, and communication cost, while also examining implementation sensitivity through multiple implementations and matched persistence configurations.

The results show that performance differences are strongly condition-dependent. Replica count and client concurrency produce the most pronounced differences, while read/write composition and conflict rate produce weaker or implementation-specific effects. The evaluated Raft configuration concentrates request processing at the elected leader and exhibits a distinct recovery gap following leader failure, whereas the evaluated EPaxos configurations distribute proposal processing across replicas. However, the magnitude of these differences depends substantially on implementation and persistence choices.

Matched persistence experiments can reverse the performance ordering observed under the baseline configuration, while the communication experiments show that the measured delay sensitivity depends on the mechanism used to impose the delay and on implementation-level message processing. These results therefore do not support a universal performance ranking between Raft and EPaxos. Instead, they show that observed differences emerge from the interaction between consensus structure, implementation, workload, communication conditions, and failure mechanisms.

The study consequently argues that empirical comparisons of consensus protocols should distinguish architectural effects from implementation effects and evaluate performance across controlled operating conditions.

Keywords

consensus, Raft, EPaxos, leader-based ordering, leaderless ordering, benchmarking

1 Introduction

Consensus protocols coordinate replicas so that a replicated state machine can apply operations consistently despite concurrent execution and replica failures. Two widely studied design approaches differ in how they establish this coordination. Leader-based protocols such as Raft [6] direct client operations through an elected leader, whereas leaderless protocols such as EPaxos [5] allow replicas to propose operations

independently and determine the dependencies required for consistent execution.

These architectural differences motivate empirical comparison, but a direct protocol-level ranking is difficult to interpret. Measured throughput and latency depend on replica count, client concurrency, workload composition, conflict rate, and communication conditions. They are also affected by implementation mechanisms including persistence, request processing, scheduling, transport, and message handling. The presence or absence of a leader is therefore an important structural property, but it does not by itself determine the performance or failure behaviour of a concrete implementation, so a measured difference between two implementations cannot automatically be attributed to the consensus algorithm alone.

In this paper, we experimentally compare Raft and EPaxos as representative leader-based and leaderless consensus protocols, measuring throughput, latency, resource utilisation, and failure behaviour under controlled variation of system and workload conditions.

We investigate the following research questions:

- **RQ1:** How do throughput and latency change with workload and system conditions, including read/write ratio, client concurrency, conflict rate, and replica count?
- **RQ2:** How do the evaluated Raft and EPaxos implementations differ in failure behaviour and in their sensitivity to communication cost?
- **RQ3:** To what extent are the observed performance differences between Raft and EPaxos attributable to implementation choices rather than to the protocol families themselves?

We contribute a reproducible experimental framework that executes both protocol families in a common containerised environment using a shared client harness and workload generator. The complete benchmark and analysis artefacts are released at <https://github.com/Sephy314/Leader-consensus-comparison>.

We further contribute a sensitivity-oriented evaluation rather than a single-configuration benchmark, varying replica count, client concurrency, read/write composition, conflict rate, failure mode, and communication conditions.

Finally, we provide an implementation-level analysis using multiple implementations of the two protocol families and matched persistence configurations, showing that implementation mechanisms can materially change the measured performance relationship and, under matched storage conditions, reverse the ordering observed in the baseline configuration.

The experiments are conducted in a local containerised environment, so the reported values are measurements of the evaluated implementations under the specified experimental conditions rather than universal estimates of production performance.

The remainder of the paper reviews related work (Section 2), describes the experimental methodology (Section 3), presents and interprets the results (Sections 4 and 5), and discusses threats to validity and conclusions (Sections 6 and 7).

2 Background and Related Work

2.1 Leader-Based and Leaderless Consensus

Leader-based protocols such as Raft [6] designate one replica as the leader: client requests are directed to it, it appends operations to its log and replicates them to followers, and an entry is committed once a quorum has acknowledged it. This centralises the write path and makes leader failure require an election before processing resumes. Paxos [4] established the foundational agreement framework, but its abstract presentation leaves substantial implementation decisions to system designers, so systems in the same protocol family can differ considerably in execution path, storage, and communication behaviour.

Leaderless protocols such as EPaxos [5] let any replica propose operations and determine dependencies among them. Commands that do not conflict can commit without a single global ordering point, at the cost of dependency tracking and conflict detection: replicas must still agree on dependencies and execute conflicting operations in a compatible order. The loss of one replica does not require electing a replacement before the others continue proposing.

2.2 Related Work

Comparative studies of consensus designs have reached different conclusions depending on the systems and workloads examined. Howard and Mortier found that the choice of implementation and configuration often matters more than the protocol family itself when comparing Paxos and Raft [3]. van Renesse et al. showed that the relative performance of Paxos and leaderless replication depends on the workload, particularly on command conflict [9]. Tollman et al. re-evaluated EPaxos under geo-replicated workloads and found that the original benchmarking approach did not expose the performance impact of conflicting commands, with substantially worse tail latency than the reported median [7]. These results motivate evaluating multiple independent implementations of each protocol family under controlled conditions, as we do here.

Our study compares two Raft implementations (HashiCorp Raft and etcd Raft) with two EPaxos implementations (the original `efficient/epaxos` and `nvanbenschoten/epaxos`) while varying replica count, concurrency, communication delay, and failure conditions, so the results describe concrete implementations under a controlled workload rather than protocol-independent properties of Raft or EPaxos.

3 Experimental Setup

3.1 Consensus Lab Architecture

The experiments are conducted with the Consensus Lab, a reproducible benchmark infrastructure that runs the evaluated implementations in a common containerised environment. Each run is an independent Docker Compose project containing one coordination service (the upstream EPaxos master [5] or a lab-provided Raft master), one container per replica, and one benchmark client. All containers share the same runtime image and the configured CPU and memory allocations.

Both protocols expose the same benchmark-facing request interface based on the upstream EPaxos `generic smr` wire protocol (`PROPOSE` $\rightarrow$ `ProposeReplyTS`). A single client binary and workload generator are used for both protocols, so the operation mix, key-selection process, concurrency control, and request-generation logic are shared. Request routing differs by protocol architecture: Raft requests are directed to the current leader reported by the raft master, whereas EPaxos requests are distributed round-robin across replicas. Reads are submitted through the consensus path in both protocols; no Raft-specific local-read optimisation is enabled.

3.2 Implementation

Each protocol is represented by two independent implementations. The lab adapters add only orchestration, monitoring, and the communication-delay injection described below; they never implement or modify consensus logic.

Raft is provided by `github.com/hashicorp/raft` v1.7.3 [2], wrapped in a thin adapter (`lab/adapters/raft/`) that converts benchmark requests into state-machine commands, invokes `raft.Raft.Apply`, and returns the response. Leader election, log replication, quorum, commitment, and term management are performed by the library. Replicas use BoltDB-backed persistent state in per-replica Docker volumes, with a snapshot threshold of 8,192 entries and 1,024 trailing logs.

The second Raft implementation uses `go.etcd.io/raft` v3.7.0 [1], wrapped in a lab adapter that follows the library’s asynchronous-storage-writes interface: a dedicated append thread batches log entries into a single write-ahead-log write with one `fsync`, mirroring HashiCorp Raft’s group commit. The adapter sets the message-size and in-flight limits the library requires, enables pre-vote to match HashiCorp Raft, and compacts its log below the applied index; the exact values are documented with the adapter in the repository.

EPaxos uses the upstream `efficient/epaxos` server and master [5], pinned at commit 791b115 and executed with `-e -exec -dreply`. Replica state is in-memory. The upstream implementation fixes the dependency-set size at five; the lab extends this field to support up to nine replicas through a documented wire-format extension that does not alter the protocol phases, quorums, or decision rules. The second EPaxos implementation uses `nvanbenschoten/epaxos` [8], pinned at commit 425bd36, wrapped in a lab adapter (`lab/adapters/nvbepaxos/`) that bridges the library’s gRPC peer transport to the benchmark’s client protocol. EPaxos state is in-memory in both implementations.

3.2.1 Communication-delay injection. The communication-cost family adds a fixed one-way delay to every inter-replica message, implemented in each implementation’s send path: HashiCorp Raft’s transport wrapper sleeps before forwarding `AppendEntries` and `RequestVote`; the etcd and nvb adapters’ per-peer sender goroutines sleep before writing; and the original EPaxos `genericsmr` send path sleeps before each outbound message (the one place the lab modifies that binary beyond orchestration). The delay is applied per message, so an implementation that sends more messages per request, or serialises its sends, is affected more strongly; the etcd adapter’s per-peer sender sleeps on its critical path. The communication-cost results should therefore be read as measurements of each implementation’s send-path structure under added delay, not as a protocol-level latency-sensitivity ranking.

Because the implementations use their native storage and execution mechanisms, the measurements evaluate complete implementations rather than isolated consensus kernels.

3.3 Client Harness and Workload Model

The benchmark client generates requests over a keyspace of 1,000 keys with a configurable read/write composition. The client is closed-loop: one worker per unit of configured concurrency issues its next request only after the previous one completes, so the configured concurrency is the maximum number of in-flight requests. For conflict experiments, a configured conflict ratio determines the probability that a request selects from a shared hot-key set; the realised hot-key fraction is measured from the generated sequence rather than assumed. Workload seeds are recorded for every repetition. Each run begins with a warm-up phase followed by a measured phase; warm-up requests are not included in performance measurements.

3.4 Metrics

For each request, the client records the run identifier, protocol, replica count, workload parameters, concurrency, operation, key, target replica, start and completion timestamps, latency, and success or failure status. The analysis pipeline derives throughput, success rate, and p50/p95/p99 latency. Per-replica resource telemetry is collected at approximately 200 ms resolution: CPU from `cgroup v2 cpu.stat`, network RX/TX from `/proc/<pid>/net/dev`, and resident memory from `/proc/<pid>/status`. Raw samples are retained per replica.

3.5 Testbed Configuration and Parameters

Table 1 lists the configuration of every reported run. To examine sensitivity to communication delay, each outbound inter-replica message can be delayed by $d = c + U(0, cj/100)$, where c is a fixed one-way cost in milliseconds and j a jitter percentage; this models application-level message delay rather than a physical network, and does not reproduce queueing, packet loss, or link-level scheduling.

Table 1: Testbed configuration. These values apply to every reported run unless the text says otherwise.

Parameter	Value
Warm-up / measured phase	3 s / 20 s
Repetitions	10 per condition, fresh Compose project each
Replica container	2.0 CPUs, 512 MB, GOMAXPROCS=4
Client container	2.0 CPUs, 512 MB, GOMAXPROCS=4
Coordination service	1.0 CPU, 256 MB
Request timeout	2,000 ms
Raft heartbeat / election	1,000 ms / 2,000 ms
Storage	Raft persistent, EPaxos in memory
Delay model	$d = c + U(0, cj/100)$
Costs c / jitter j	0, 1, 3, 5, 10 ms / 0, 5, 10, 50, 100 %
Isolation	fresh containers, networks and volumes per repetition

Each repetition uses a fresh Docker Compose project with new containers, networks, and volumes. The harness verifies before execution that no leftover containers, networks, volumes, or ports remain, and verifies teardown afterwards; a run that fails those checks is recorded as an invalid attempt rather than silently reused. Configurations are scheduled by blocked randomisation: each repetition block contains every configuration exactly once in a seeded shuffled order, and each repetition uses an independent recorded workload seed.

3.6 Experiment Suite

Table 2 lists the experiment families. The first seven form the recorded suite and the last three were added afterwards, each analysed from its own results subtree and never pooled with the recorded dataset (Sections 4.7–4.9). The Workload and Concurrency families overlap in concurrency by design: the Workload family measures the interaction between write ratio and concurrency, while the Concurrency family isolates the concurrency dimension at 100 % writes and extends it to 256, where the throughput difference is largest. That overlap also provides a check on run-to-run variability. The Election family uses the *measured* failed-election count from the adapters’ pre-vote and vote counters rather than the configured target.

Failure injection is deterministic: the runner kills the selected container with `docker kill` at a configured offset into the measured phase. For Raft leader failure, the current leader is obtained from the raft master and mapped to its container; the client reconnects to the new leader. For EPaxos replica failure, replica 0 is killed and the client moves to the remaining replicas. Recovery intervals are reconstructed from request timestamps and the approximately 200 ms monitoring resolution and are therefore approximate.

3.7 Correctness Validation

The performance experiments are not a formal verification of consensus correctness. A separate harness (`runner correctness`) issues 2,000 deterministic requests, each writing a unique key, and compares replica state snapshots returned by a read-only `GetState` RPC, so it does not assume that the two protocols expose equivalent log representations. Each implementation is tested at concurrency 1, 8, and 32 and under the failure scenarios of the failure family, including a restarted Raft

Table 2: The experiment families, each with ten independent repetitions. The first seven form the recorded suite; the last three were added afterwards and are analysed from their own results subtrees, never pooled with it. The defaults of Section 3 are three replicas, 100 % writes and concurrency 32.

Family	Varied
Workload	write ratio 0–100 %, concurrency 1–128
Scaling	replicas 3–9
Conflict	configured conflict ratio 0–100 %
Concurrency	concurrency 1–256
Communication cost	one-way delay 0–10 ms; jitter 0–100 % at 5 ms
Election	failed-election target 0–5
Failure	Raft leader, Raft follower, EPaxos replica kill
Controlled network delay	tc/netem delay 0–10 ms
Persistence matching	storage mode, memory against durable
Conflict validation	conflict ratio $\times$ number of hot keys

replica recovering its committed state from its persistent log. This checks state convergence for the tested scenarios; it is not a general linearisability proof.

3.8 Experimental Integrity and Statistical Analysis

Host-level telemetry (load average, available memory) is recorded at approximately 200 ms resolution for every run, and host suspension, scheduling gaps, CPU starvation, and clock anomalies are detected and recorded. A run flagged as contaminated is excluded from aggregates and replaced, so each configuration keeps its planned number of observations; the replacement is a new attempt and the original remains in the raw dataset.

All aggregates are computed from raw per-run observations. Because a small number of repetitions stalled (Section 4), throughput and latency are summarised by the median of the per-repetition values, with 95 % confidence intervals from a non-parametric bootstrap (10,000 resamples). Protocol differences use the exact Mann-Whitney U test and workload-trend hypotheses the Kruskal-Wallis test, both without a multiplicity correction: a quoted p -value is the uncorrected value for that single comparison. The election experiment is treated as a Raft-specific recovery characterisation reported descriptively, with no causal interpretation assigned to the measured association.

4 Evaluation

Unless otherwise stated, reported point estimates are medians over ten independent repetitions, results use three replicas, 100 % writes, and client concurrency 32, and results describe the evaluated implementations under the experimental configuration of Section 3. All figures are generated from the processed benchmark data by `lab/report/figures.py` and

show medians across repetitions, with individual repetition values overlaid where available. Differences should not be interpreted as protocol-level causal effects unless the experiment directly isolates the corresponding mechanism.

4.1 Throughput and Scalability

Figure 1 shows measured throughput as the number of replicas increases. Throughput decreased with replica count for both evaluated implementations. At three replicas, the measured median was 5,291 req/s for EPaxos and 4,350 req/s for Raft. At five replicas, the medians were 5,442 and 3,148 req/s, respectively, while at nine replicas they were 2,002 and 2,112 req/s (Table 3). Relative to the three-replica measurement within this experiment family, the nine-replica median was 62 % lower for EPaxos and 51 % lower for Raft. The ratio of the two medians was 1.22 at three replicas, 1.73 at five, 1.12 at seven, and 0.95 at nine replicas. Thus, the measured throughput difference varied with replica count.

We report medians rather than means because a small number of repetitions stalled. Across the entire dataset (1,302 runs), eight repetitions completed fewer than 10 % of the requests of their configuration’s median: two Raft runs (0.3 % of Raft runs) and six EPaxos runs (1.0 % of EPaxos runs). Six of the eight were slow from start to finish with no failed requests, while two EPaxos runs, at seven and nine replicas, showed persistent request timeouts. The two Raft stalls coincided with elevated host load (load average 7.0–8.6 against 3.8–4.0 for healthy runs of the same configuration); the six EPaxos stalls occurred at both low and high host load. None of the eight runs was flagged by the host-telemetry contamination rule, so they remain in the dataset. The observed stall counts (2 of 680 Raft runs and 6 of 610 EPaxos runs) were not significantly different (Fisher’s exact test, $p = 0.16$).

A second anomaly affects EPaxos at seven and nine replicas: request timeouts. Eighteen of the twenty EPaxos runs at those sizes had 30–247 requests time out at the 2,000 ms client timeout (success rates 98.6–99.9 %); the other two completed every request, and Raft runs at the same sizes had no timeouts. Each timeout occupies a client worker for the full 2 s, reducing the concurrency available to successful requests. For a closed-loop client with concurrency N , the accounting

$$N \approx X_{\text{ok}}W_{\text{ok}} + X_{\text{timeout}}W_{\text{timeout}},$$

where $X_{\text{timeout}} = N_{\text{timeout}}/W$ and $W_{\text{timeout}} = 2$ s, with W the measured phase window (20.0–22.0 s in these runs), is consistent with every EPaxos run at these sizes to within 0.8 % of N . The successful requests themselves completed in 5.4–5.6 ms at p50, similar to the three-replica measurements, so the reduced throughput is accounted for by requests occupying workers until the timeout rather than by slower successful requests. The available instrumentation does not establish the specific execution-path mechanism responsible for the timeouts.

The same nominal configuration (three replicas, 100 % writes, and concurrency 32) also occurs independently in

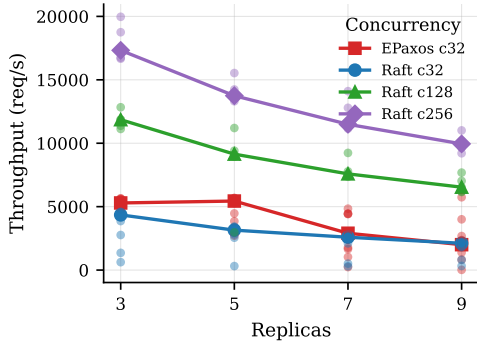


Figure 1: Throughput by replica count at 100 % writes. EPaxos at client concurrency 32 (the recorded comparison) and Raft at concurrency 32, 128, and 256. The replica-scaling penalty persists at every concurrency level. Points show individual repetitions; lines mark the medians.

several other steady-state experiment families, providing an indication of run-to-run variability. The four family medians were 4,350, 4,367, 4,482, and 4,440 req/s for Raft, and 5,291, 5,287, 5,175, and 5,249 req/s for EPaxos. The EPaxos medians differed by approximately 2 % across these measurements, and the Raft medians by approximately 3 %. Cross-family variability was therefore of similar magnitude for the two implementations.

For the scaling family, the difference between the implementations was statistically significant at three and five replicas (exact Mann–Whitney U test, uncorrected $p < 0.001$), while no statistically significant difference was detected at seven ($p = 0.25$) or nine ($p = 0.68$) replicas. The 95 % bootstrap confidence intervals of the medians overlap at seven and nine replicas (Table 3). The measured EPaxos/Raft throughput difference was therefore statistically significant only at the smaller cluster sizes in this experiment.

A Raft-only follow-up at client concurrency 128 and 256 (five repetitions per cell, 40 runs) found the replica-scaling penalty persists: the nine-replica median was 45 % lower at c128 (11,856 to 6,535 req/s) and 43 % lower at c256 (17,325 to 9,949 req/s), versus 51 % at c32 (Figure 1); the r3-to-r9 difference was significant at both levels ($p = 0.008$, exact Mann–Whitney U , $n = 5$ per group). The leader’s CPU utilisation grew with concurrency (about 53 % at c32 to 82 % at c128 and 103 % at c256 at three replicas) while followers stayed below 35 %. The leader remained the CPU hotspot under higher offered concurrency, approaching but not exceeding its two-core allocation, so these measurements do not demonstrate a hard CPU capacity ceiling.

Figure 2 shows throughput across the client concurrency sweep. Throughput increased with concurrency for both implementations over the tested range. At concurrency 1, the measured medians were 304 req/s for Raft and 175 req/s for EPaxos. At concurrency 256, the medians were 19,278 req/s

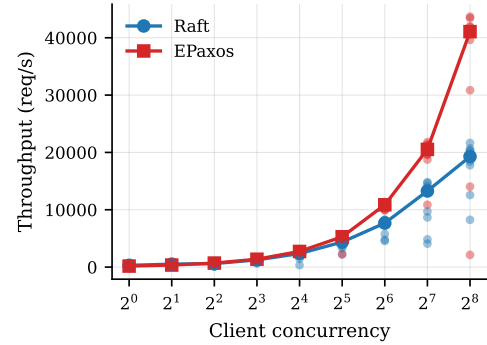


Figure 2: Median throughput by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions.

and 41,069 req/s, respectively. The largest individual observations at concurrency 256 were 21,656 req/s for Raft and 43,681 req/s for EPaxos.

Between concurrency 64 and 256, the Raft median increased from 7,673 to 19,278 req/s, while the EPaxos median increased from 10,862 to 41,069 req/s. Both implementations therefore continued to gain throughput as concurrency increased, although the magnitude of the increase differed between the two implementations.

Figure 3 shows throughput across the write-ratio sweep. EPaxos medians ranged from 5,165 to 5,461 req/s, while Raft medians ranged from 4,219 to 4,474 req/s. Neither implementation exhibited a monotonic or otherwise consistent throughput trend across the tested write ratios. The highest EPaxos median occurred at 10 % writes, approximately 4 % above its 100 % write measurement, and the highest Raft median also occurred at 10 % writes.

The variation across write ratios was not statistically significant. A Kruskal–Wallis test of the five write-ratio groups gave $H = 1.68$ for Raft and $H = 1.75$ for EPaxos, both below the χ^2_4 critical value of 9.49. The measurements therefore do not provide evidence of a systematic throughput effect associated with write ratio under the tested conditions.

Figure 3 shows throughput across the configured conflict-ratio sweep. Median throughput ranged from 4,149 to 4,512 req/s for Raft and from 5,078 to 5,391 req/s for EPaxos. A Kruskal–Wallis test of the seven conflict-ratio groups gave $H = 4.44$ for Raft and $H = 3.13$ for EPaxos, both below the χ^2_6 critical value of 12.59. No monotonic throughput relationship with the configured conflict ratio was observed for either implementation.

4.2 Latency

Figure 4 shows median (p50) latency as the number of replicas increases. At three replicas, the median latency was 6.9 ms for Raft and 5.6 ms for EPaxos. Raft latency increased to 9.2 ms at five replicas and 12.0 ms at seven replicas, reaching 14.6 ms at nine replicas. EPaxos remained within a relatively

Table 3: Throughput and median (p50) latency by replica count, at 100 % writes and concurrency 32. (a) Throughput in req/s, with the 95 % bootstrap confidence interval of the median; medians are used because several repetitions stalled (Section 4), and the mean is lower for Raft at every replica count and for EPaxos at five and seven replicas. (b) Latency in milliseconds, the median of the ten per-repetition p50 values.

(a) throughput (req/s)						(b) p50 latency (ms)		
Replicas	Raft	95 % CI	EPaxos	95 % CI	EPaxos/ Raft	Replicas	Raft	EPaxos
3	4,350	[2,610–4,494]	5,291	[5,117–5,598]	1.22×	3	6.9	5.6
5	3,148	[2,770–3,346]	5,442	[4,468–5,610]	1.73×	5	9.2	5.5
7	2,589	[1,522–2,608]	2,903	[1,404–4,435]	1.12×	7	12.0	5.4
9	2,112	[1,337–2,159]	2,002	[1,318–3,183]	0.95×	9	14.6	5.5

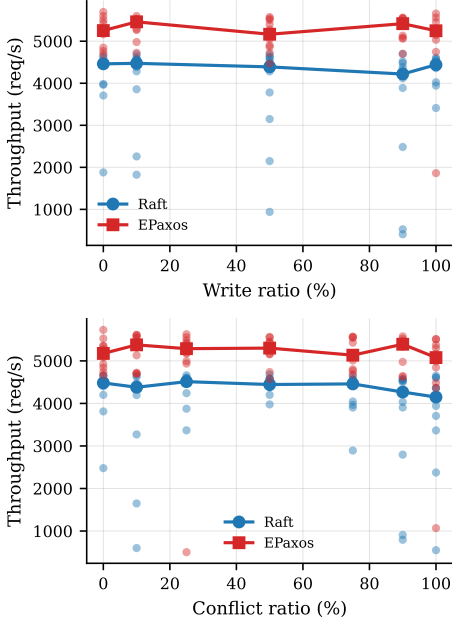


Figure 3: Throughput at 3 replicas and concurrency 32. (a) By write ratio; (b) by configured conflict ratio at 100 % writes. Neither sweep shows a systematic throughput trend. Points show individual repetitions; lines mark the medians.

narrow range of 5.4–5.6 ms across the tested replica counts (Table 3b).

The Raft p50 therefore increased monotonically with replica count, whereas the EPaxos p50 remained within the 5.4–5.6 ms range.

The p50 values should be read together with the tail: the p99 was 21.4 ms for Raft and 23.8 ms for EPaxos at three replicas, increasing to 26–29 ms for Raft at seven to nine replicas while remaining at 14–16 ms for EPaxos. The median of the per-repetition p50 values reduces the influence of the stalled repetitions described earlier.

Figure 5 shows median latency across client concurrency. At concurrency 1, median latency was 2.8 ms for Raft and 5.5 ms for EPaxos. At concurrency 32, the corresponding

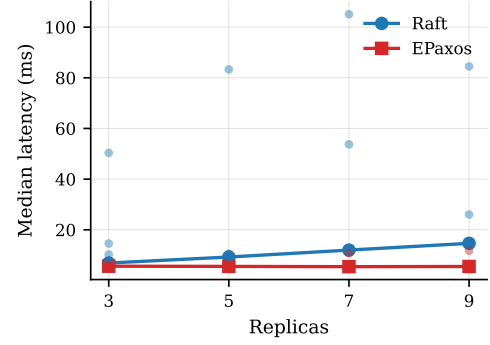


Figure 4: Median (p50) latency by replica count at 100 % writes and concurrency 32. Points show individual repetitions; the line marks the median.

values were 6.9 ms and 5.6 ms. At concurrency 256, they were 12.1 ms and 5.6 ms.

The relative latency ordering therefore changed with concurrency: Raft had the lower median at concurrency 1, while EPaxos had the lower median at concurrency 32 and above.

As with throughput, the same nominal configuration (three replicas, concurrency 32) is measured by several experiment families. With medians, the p50 was consistent across families: Raft measured 6.9 ms in the scaling family and 6.9 ms in the concurrency family, while EPaxos measured 5.6 ms in both.

For a closed-loop client, Little’s law relates mean latency to concurrency and throughput as $W = N/X$. The reported p50 is the median rather than the mean, so it need not equal this quantity. At concurrency 256, the Raft p50 of 12.1 ms was close to N/X ($256/19,278 \approx 13.3$ ms), while the EPaxos p50 of 5.6 ms was below N/X ($256/41,069 \approx 6.2$ ms). This difference is compatible with a response-time distribution whose median is below its mean. We report p50, p95, and p99 in the processed dataset, using p50 here as the central-tendency summary.

At seven and nine replicas, the EPaxos p50 of 5.4–5.5 ms was substantially below N/X ($32/2,002\text{--}2,903 \approx 11.0\text{--}16.0$ ms). As described in Section 4, this difference is accounted for by the request timeouts at those sizes: each timeout occupies a worker for the full 2 s, and the closed-loop concurrency accounting including the timeout term is consistent with every

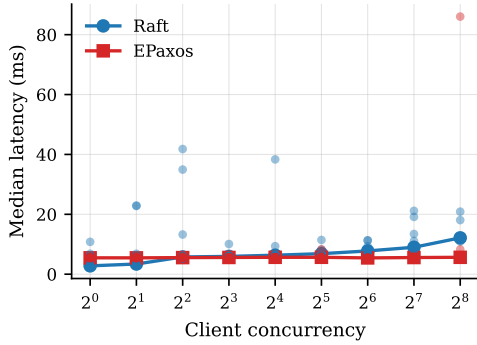


Figure 5: Median (p50) latency by client concurrency at 3 replicas and 100 % writes. Points show individual repetitions; the line marks the median.

EPaxos run. Raft runs at the same sizes had no timeouts, and their measured N/X values were closer to their reported p50 latencies.

4.3 Resource Utilisation

Figure 6 summarises the per-replica resource measurements. Across the evaluated runs, mean per-replica CPU utilisation was 36.5 % for Raft and 35.4 % for EPaxos. Mean peak per-replica RSS was 28.3 MB for Raft and 46.4 MB for EPaxos. Mean per-replica network rates were 451 kB/s RX and 454 kB/s TX for Raft, compared with 410 kB/s RX and 394 kB/s TX for EPaxos.

Figure 7 examines the distribution of CPU utilisation in the nine-replica scaling experiment. Raft per-replica means ranged from 16.3 % to 29.4 %. The replica serving as leader during a given repetition consumed substantially *more* CPU than the followers. Taking the leader from each run’s own Raft log, the leader/follower ratio of mean utilisation was 2.2 at three replicas, 2.8 at five, 3.3 at seven and 3.8 at nine replicas, using the medians over the ten repetitions, so the leader’s CPU cost grew relative to each follower as the cluster grew. (The leader’s share of the cluster’s total CPU falls as replicas are added; it is the per-follower ratio that rises.)

This is consistent with the request path of the evaluated HashiCorp Raft implementation: the leader serves the benchmark client, applies every command, persists its own log, and broadcasts to every peer, whereas a follower receives, persists, and applies. A CPU profile of one three-replica run shows the same shape of work on both roles, about two thirds of the samples going to system calls and lock waits on the persistence path, with the leader differing from a follower chiefly in how much of that work it performs. The instrumentation does not separate the persistence, application, and transport contributions, but the direction is unambiguous: the elected leader was the CPU hotspot in the evaluated implementation, and its relative cost grew with the number of peers it serves.

Table 4: Failure behaviour at 3 replicas, 100 % writes, concurrency 32. Gap is the largest interval with no successful write completion after failure injection.

Failure	Protocol	Gap (s)	Failed req.
Leader kill	Raft	1.81–4.74	2,169–4,133
Follower kill	Raft	0.02–0.04	0
Replica kill	EPaxos	0.02–0.09	10–106

4.4 Failure Recovery Behaviour

Failure outcomes differed by failure type (Table 4, Figure 8). Killing the Raft leader produced a write-availability gap of 1.81–4.74 s and 2,169–4,133 failed requests. Killing a Raft follower produced a gap of 0.02–0.04 s with no failed requests. Killing an EPaxos replica produced a gap of 0.02–0.09 s with 10–106 failed requests.

All runs completed successfully according to the experiment harness; the reported failed requests correspond to requests issued during the transition and are distinct from run-level failure.

The longer availability gap following Raft leader termination coincided with the configured Raft election and request-retry behaviour. In contrast, the tested follower-failure case did not require replacement of the Raft leader, while the EPaxos replica-failure case continued with the remaining replicas. These results describe the tested failure scenarios and should not be interpreted as representative of all leader-based or leaderless implementations.

The election experiment provides a more controlled characterisation of Raft-specific recovery under repeated election disruption. The measured number of failed elections ranged from 0 to 13 across the 60 runs. The availability gap was strongly associated with the measured election count ($r = 0.90$, $p < 10^{-4}$), but this relationship is partly mechanical because the isolation interval itself increases with the configured disruption duration.

To examine recovery independently of the imposed isolation interval, the analysis also measures the time from the end of the isolation window to the first successful request. This post-isolation recovery interval was more variable and showed a weaker positive association with the measured failed election count ($r = 0.41$, $p = 0.001$, $n = 60$). This result is reported as an observed association rather than as evidence of a causal relationship.

4.5 Correctness Validation

The correctness harness described in Section 3 checks state convergence under the tested workloads and failure scenarios. All 18 tests passed (Table 5): every replied request was applied exactly once on every replica, the surviving replicas agreed on the final state, the restarted Raft replicas recovered the committed state from their persistent logs, and requests issued after each fault were processed normally. Because the correctness workload assigns a unique key to each request,

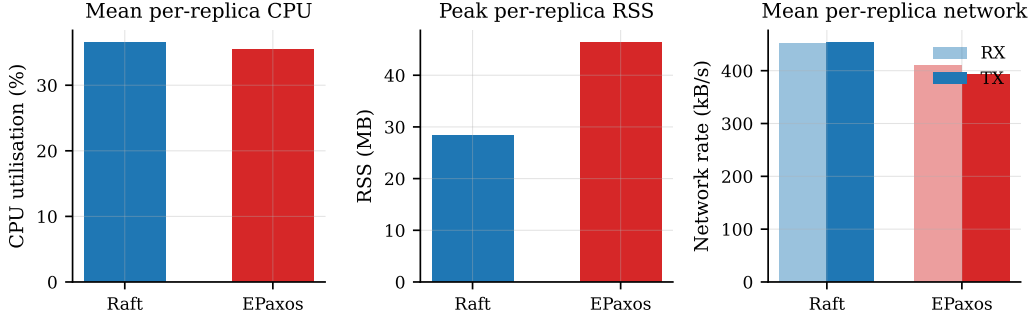


Figure 6: Mean per-replica CPU utilisation, peak per-replica RSS, and mean per-replica network rates, aggregated over the evaluated runs.

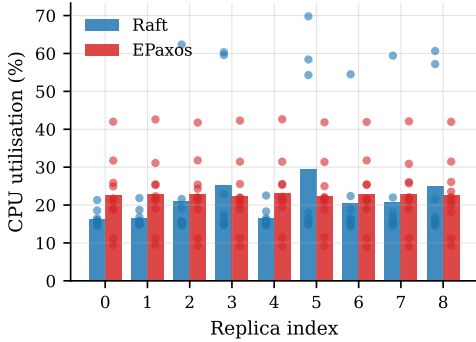


Figure 7: Per-replica CPU utilisation in the 9-replica scaling runs. Bars show means over repetitions; points show individual repetitions.

Table 5: Correctness-validation results: 2,000 deterministic requests per test at 3 replicas, 100 % writes. Every replied request was applied exactly once on every replica, surviving replicas agreed on the final state, and requests issued after each fault were processed normally.

Implementation	Sequential	c=8	c=32	Fault
Raft (HashiCorp)	PASS	PASS	PASS	follower/leader
Raft (etcd)	PASS	PASS	PASS	follower/leader
EPaxos (original)	PASS	PASS	PASS	replica
EPaxos (nvb)	PASS	PASS	PASS	replica

this is primarily a state-convergence check rather than an independent verification of operation-order semantics.

4.6 Communication Cost

The communication-cost family evaluates sensitivity to additional application-level one-way message delay and should be interpreted as a controlled communication-condition experiment rather than a direct simulation of a physical network.

Figure 10a shows mean throughput at each delay level with zero jitter for four implementations. The implementations exhibited substantially different sensitivity to the injected delay. HashiCorp Raft decreased from 4,502 req/s at zero delay to 1,280 req/s at 10 ms, a 72 % reduction. nvb EPaxos decreased from 4,346 to 833 req/s, an 81 % reduction, while the original EPaxos implementation decreased from 5,527 to 221 req/s, a 96 % reduction. The etcd Raft implementation showed a different response: its throughput decreased from 6,420 req/s at zero delay to 375 req/s at 1 ms and 45 req/s at 10 ms.

The measured ordering therefore differed across implementations under added communication delay. The 95 % confidence intervals of the means were separated between the evaluated implementations at every non-zero delay level.

Median latency increased with the injected delay for all four implementations. At 10 ms, the measured p50 values were 22.6 ms for HashiCorp Raft, 36.2 ms for nvb EPaxos, 125.9 ms for original EPaxos, and 703.4 ms for etcd Raft. The implementation ordering observed for latency at this condition corresponded to the throughput ordering.

The jitter sweep produced a smaller change than the corresponding increase in fixed delay. At a 5 ms base delay, increasing jitter from 0 to 100 % reduced throughput from 2,316 to 1,760 req/s for HashiCorp Raft (24 %), from 1,432 to 1,082 req/s for nvb EPaxos (24 %), from 479 to 305 req/s for original EPaxos (36 %), and from 87 to 59 req/s for etcd Raft (32 %).

Under the implemented jitter model, 100 % jitter also raises the mean injected delay from 5 to 7.5 ms, so the experiment cannot separate delay variance from that increase in mean cost.

4.7 Persistence Matching

The recorded baseline compares Raft implementations that persist their logs to storage with EPaxos implementations that keep replica state in memory (Section 6). The persistence-matching family holds workload, replica count, concurrency and network constant and varies only the storage mode, using only modes the implementation itself provides. It is

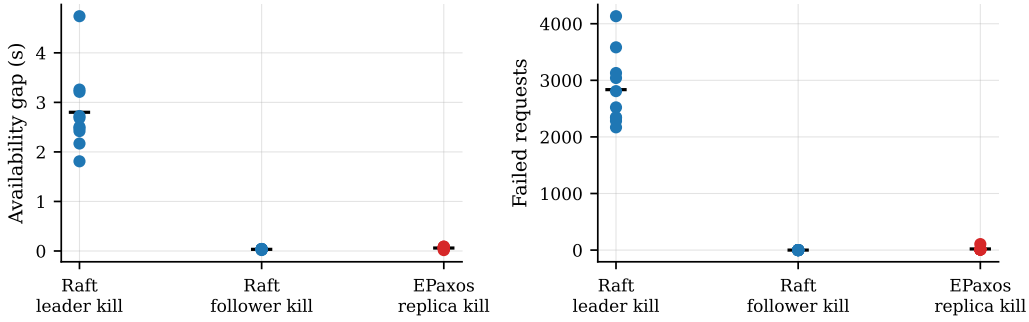


Figure 8: Write-availability gap and failed requests by failure type at 3 replicas, 100 % writes, and concurrency 32. Points show individual repetitions; the bar marks the mean.

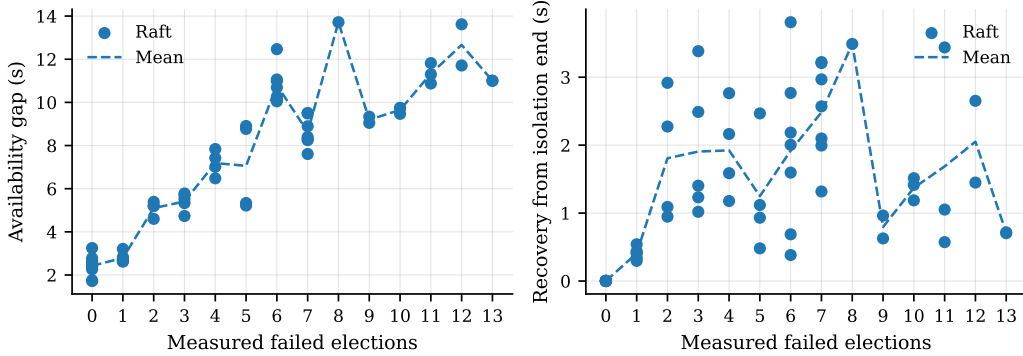


Figure 9: Election-failure recovery in Raft at 3 replicas, 100 % writes, and concurrency 32. Left: write-availability gap versus the measured number of failed elections. Right: recovery time from the end of the isolation window to the first successful request. Points show individual repetitions; the dashed line marks the mean per value.

analysed from its own results subtree and never pooled with the recorded dataset; 140 of its 147 runs were included, and every condition retains its planned ten repetitions after the replacements described at the end of Section 4.9.

Table 6 and Figure 11 show throughput at three and five replicas. The two modes that reproduce the recorded baseline agree with it: at three replicas the durable HashiCorp Raft median of 4,425 req/s is 1.7 % above the recorded 4,350 req/s, and the in-memory original EPaxos median of 5,585 req/s is 5.6 % above the recorded 5,291 req/s, the upper edge of the 2–3 % cross-family spread reported above.

Matching the storage mode changes the direction of the comparison at both replica counts. With both implementations in memory, HashiCorp Raft measured 29,660 req/s against 5,585 req/s for the original EPaxos (5.3 $\times$); with both durable, 4,425 against 2,005 (2.2 $\times$). In the second pair, etcd Raft measured 12,884 req/s against 4,226 req/s for nvb EPaxos in memory at three replicas (3.0 $\times$) and 8,437 against 1,600 at five replicas (5.3 $\times$). Every matched comparison in this family therefore places the Raft implementation above the EPaxos implementation, and each is significant (exact Mann–Whitney U test, $p < 0.001$). The durable cell of the

second pair has no EPaxos counterpart: the nvb EPaxos library offers only in-memory storage, and no persistence mechanism was added to fill the gap, so the figure shows that bar for Raft alone.

The storage modes account for the reversal. Relative to the same implementation’s in-memory median, durable storage retained 15 % for HashiCorp Raft, 48 % for etcd Raft and 36 % for the original EPaxos at three replicas, and 13 %, 69 % and 33 % at five replicas ($p < 0.001$ for each comparison). The recorded comparison therefore measured the implementation with the largest durability cost (HashiCorp Raft, which loses 85 % of its in-memory throughput when its log is persisted) against one whose evaluated mode is in-memory (the original EPaxos, which loses 64 % when made durable). The recorded EPaxos/Raft ratio of 1.22 is consistent with that pairing, so it should not be read as a property of the two protocol families. This family measures one workload at two replica counts, so it establishes that the recorded ordering depends on the storage modes; it does not establish that persistence-matched deployments would order the two families the same way at other replica counts or write ratios.

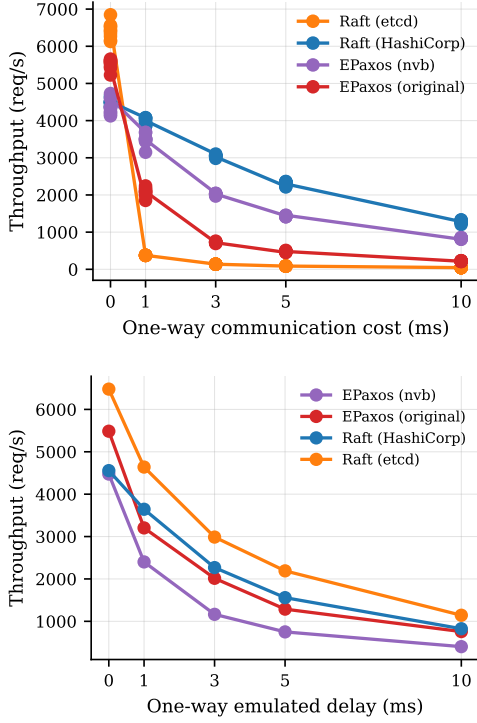


Figure 10: Throughput against added one-way delay at 3 replicas, 100 % writes and concurrency 32, for the four evaluated implementations. (a) Delay injected inside each implementation’s send path, where the implementations diverge sharply. (b) The same nominal delay applied to inter-replica traffic with `tc/netem`, leaving the implementations unmodified, where the spread is much narrower.

The p50 ordering follows the throughput ordering. At three and five replicas, durable HashiCorp Raft measured 6.8 and 9.3 ms against 0.8 and 0.9 ms in memory; durable etcd Raft 4.9 and 5.1 ms against 1.7 and 2.9 ms in memory; durable original EPaxos 14.5 and 15.6 ms against 5.7 and 5.5 ms in memory; and in-memory nvb EPaxos 7.1 and 18.6 ms.

4.8 Controlled Network Delay

The communication-cost family described above injects delay inside each implementation’s send path. The controlled network-delay family applies the same nominal condition from outside the implementations: the runner installs a `tc/netem` qdisc inside each replica’s network namespace and delays only traffic addressed to another replica, so no adapter sleeps and no consensus source is modified. The 0 ms point is the family’s own baseline, recorded without a qdisc, and all 200 runs were included.

Median throughput declined monotonically with the applied delay for all four implementations (Figure 10b); a Kruskal–Wallis test over the five delay levels gives $H = 47.06$

Table 6: Persistence matching at 3 and 5 replicas, 100 % writes and concurrency 32. Cells give the median throughput (req/s) of the evaluated Raft implementation against the evaluated EPaxos implementation of the same pair, with the Raft/EPaxos ratio in brackets. Every cell is the median of ten included repetitions. Modes are the implementations’ own: HashiCorp Raft BoltStore against NewInmemStore, etcd Raft write-ahead log against `-no-wal`, and efficient/epaxos upstream `-durable` against its in-memory default, so that cell is a gap and is never filled with an added mechanism.

Persistence	Rep.	HashiCorp Raft / original EPaxos	etcd Raft / nvb EPaxos
durable	3	4,425 / 2,005 [2.2×	gap
durable	5	3,294 / 1,838 [1.8×	gap
memory	3	29,660 / 5,585 [5.3×	12,884 / 4,226 [3.0×
memory	5	24,573 / 5,613 [4.4×	8,437 / 1,600 [5.3×

against $\chi^2_4 = 9.49$ for each implementation, and the observed range of the ten repetitions does not overlap between adjacent delay levels for any of them. At 10 ms the implementations retained 18.1 % (HashiCorp Raft), 17.6 % (etcd Raft), 13.8 % (original EPaxos) and 9.0 % (nvb EPaxos) of their 0 ms medians. The p50 latency at 10 ms was 36.9, 25.7, 44.6 and 83.0 ms, respectively, against 6.6, 4.7, 5.7 and 6.8 ms at 0 ms.

The controlled condition compresses the differences between implementations that the application-level family measured: the fraction of 0 ms throughput retained at the same nominal 10 ms spans 9.0–18.1 % across the four implementations, a factor of two, where the application-level family spanned 0.7–28.4 %, a factor of forty. The near-collapse of etcd Raft measured by the application-level family is therefore attributable to the injection mechanism, the per-message sleeps on the adapters’ send paths, rather than to added delay as such, as Section 6 anticipated. The ordering also changes: the two implementations that retained the most under application-level delay (HashiCorp Raft at 28.4 % and nvb EPaxos at 19.2 %) retain less under OS-level emulation (18.1 % and 9.0 %), consistent with the emulation delaying each inter-replica exchange in both directions rather than a single send. The two mechanisms are not interchangeable; the controlled family is the evidence used here for delay sensitivity.

4.9 Conflict Validation

The recorded conflict-ratio sweep configures how often the workload targets a hot key, but a configured fraction does not by itself establish the access pattern that was realised or the protocol-level contention it produced. The conflict-validation family re-runs the sweep while varying both the configured hot-key fraction and the number of distinct hot keys that fraction targets, and records the realised fraction from the per-request hot flag; 480 of its 492 runs were included, twelve

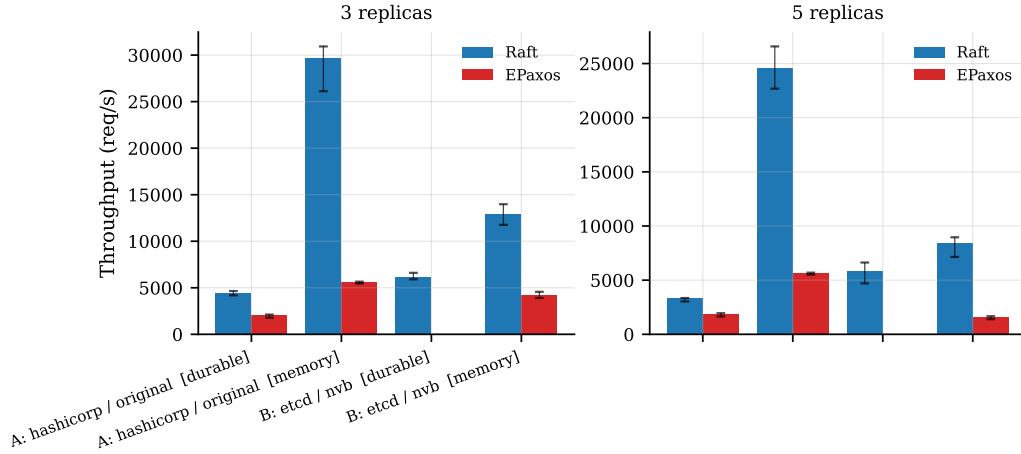


Figure 11: Median throughput (req/s) by storage mode at 100 % writes and concurrency 32, for the two implementation pairs. Bars show the median of the included repetitions; error bars show the observed range. Pair A is HashiCorp Raft against the original EPaxos and pair B is etcd Raft against nvb EPaxos. The durable cell of pair B contains a Raft bar only, because the nvb EPaxos library provides no durable mode.

excluded as three flagged attempts and their nine replaced predecessors, so all 48 conditions contain ten repetitions.

Figure 12(a) shows that the workload delivered what it configured. With a single hot key the top key’s share of requests equalled the configured fraction (9.9, 24.9, 49.7, 74.8 and 100.0 % at 10, 25, 50, 75 and 100 %); with eight hot keys it was one eighth of it, as the design intends. At 100 % the workload contains no cold keys at all, leaving one or eight distinct keys observed per run, so that point is not on the same continuum as the rest of the sweep and is reported separately below.

Throughput showed no systematic relationship with the configured fraction. For HashiCorp Raft the medians span 4,354 to 4,541 req/s across both key-set sizes, for etcd Raft 5,848 to 6,301 req/s, and for the original EPaxos 5,448 to 5,517 req/s. Kruskal–Wallis tests over the 0–75 % points give $H \leq 6.73$ against $\chi^2_4 = 9.49$ for the six combinations of these three implementations, and the p50 medians remain in 6.7–6.9 ms, 4.8–5.1 ms and 5.66–5.68 ms respectively. This reproduces the recorded null result for these implementations and shows that it is not explained by the configured fraction failing to produce the intended access pattern. The tails behave the same way over 0–75 %: the p99 medians stay within 10.4–11.4 ms for etcd Raft, 14.8–17.3 ms for HashiCorp Raft and 13.3–17.0 ms for the original EPaxos, whereas nvb EPaxos moves from 16.6 to 36.1 ms together with its median (2.04× and 2.12× at 75 % against 0 %).

nvb EPaxos is the exception. Its median throughput declined monotonically with the configured fraction over 0–75 %, from 4,123 to 2,129 req/s with one hot key and from 4,238 to 2,324 req/s with eight ($H = 42.28$ and $H = 41.84$), and its p50 rose from 7.3 to 13.9 ms and from 7.1 to 12.4 ms. At 100 % both of its medians rise above their 0 % values, to 6,103 req/s and 6,631 req/s with p50 values of 5.1 and 4.5 ms.

The reversal coincides with the degenerate keyspace of that point; the instrumentation does not establish its mechanism, and this implementation exposes no protocol counters with which to probe it further. Conflict sensitivity in this family is therefore implementation-specific: three of the four implementations show no throughput response to the configured fraction over the range in which the keyspace still contains cold keys, while the fourth declines monotonically.

Figure 12(b) reports the one protocol-level indicator available: the slow-path share of the original EPaxos implementation declines as the configured fraction rises, from 97.8 to 43.8 % with one hot key and from 97.9 to 72.5 % with eight. The `conflicted` counter moves with it, but in this implementation it counts replicas whose `PreAccept` replies disagree on an instance’s sequence number and dependencies, not key conflicts. Both counters are reset whenever a client connection is accepted, so they cover about 9.5 % of the successful requests rather than the measured phase; that coverage is near-constant across the conditions of this family, which makes the shares comparable between them but leaves them a within-window indicator rather than a census. Read with that limitation, the direction is the opposite of the expectation that a higher configured conflict fraction forces more slow-path executions. It is consistent with instances on a hotter key being serialised onto that key’s dependency chain, where replicas agree more often; the available instrumentation does not separate that reading from others, and no counter is available for nvb EPaxos.

Two replacement rules were applied to the three families, both keeping the original attempt in the raw dataset. An attempt that failed, or that host telemetry flagged as contaminated, was re-run as a new attempt. A repetition that is anomalous without being flagged is invisible to that rule, so a second rule was fixed in advance and applied uniformly

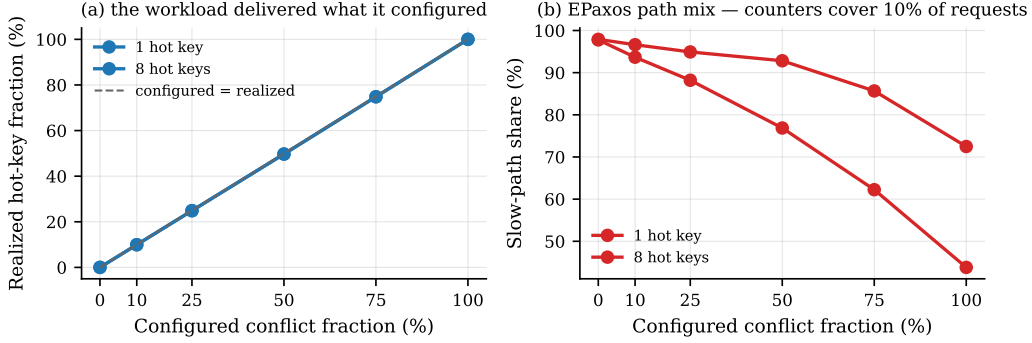


Figure 12: Conflict validation at 3 replicas, 100 % writes and concurrency 32. Left: the realised hot-key fraction against the configured one, which the per-request hot flag makes exact. Right: the EPaxos slow-path share for the one implementation that exposes path counters; the counters are reset whenever the server accepts a client connection, so they cover approximately 9.5 % of the successful requests of a run.

to every condition: an included repetition measuring below 0.7 times the median of its own configuration is re-run, and the report keeps the replacement whatever its value. Eleven repetitions met the second rule and seven attempts were replaced by the first. No included repetition of the three families now measures below 0.7 times its condition’s median, no repetition’s p50 exceeds 1.5 times that of its condition, and every condition has its planned ten observations; replacing those repetitions moved no median reported in this section by more than 3 %.

4.10 Summary of Findings

The measurements support the following observations under the evaluated conditions:

- Throughput decreased with replica count for both implementations, but the EPaxos/Raft ratio changed with cluster size, and the difference was statistically detectable only at three and five replicas.
- Eight of 1,302 repetitions stalled (two Raft, six EPaxos), motivating the use of medians. EPaxos at seven and nine replicas also showed request timeouts, which account for part of its reduction there through the closed-loop concurrency accounting of Section 4.
- Client concurrency widened the gap between the implementations rather than narrowing it: the EPaxos advantage was largest at the highest concurrency tested, while at the lowest the latency ordering of the two reversed.
- Write ratio and conflict ratio produced no systematic throughput trend (Kruskal–Wallis $H \leq 4.44$ against critical values of 9.49 and 12.59). Conflict validation showed that the configured fraction was realised exactly in the workload and that three of the four implementations still showed no response, while nvb EPaxos declined monotonically: conflict sensitivity is implementation-specific.

- Failure of the Raft leader produced an availability gap two orders of magnitude larger than failure of a Raft follower or an EPaxos replica.
- The elected leader was the CPU hotspot in the evaluated HashiCorp Raft implementation, using 2.2 times a follower’s CPU at three replicas and 3.8 times at nine, while the two protocol families used similar per-replica CPU overall.
- Implementation choices change the measured relationship. The spread of retained throughput differs by about a factor of twenty between the two delay mechanisms (9.0–18.1 % against 0.7–28.4 %); matching the storage mode reverses the recorded ordering at both replica counts tested; and the recorded EPaxos advantage at three replicas reflects which storage mode each implementation was run in.

5 Discussion

5.1 RQ1: Throughput and Latency Across Workloads

RQ1 asks how the measured throughput and latency of the evaluated leader-based and leaderless implementations change with replica count, client concurrency, conflict rate, and read/write composition. The clearest differences were observed for replica count and client concurrency, whereas the tested conflict and write ratios produced comparatively little systematic variation.

Increasing the replica count reduced throughput for both implementations, but the EPaxos/Raft ratio changed with cluster size rather than remaining constant, and the difference was statistically detectable only at the three- and five-replica configurations (Section 4, Table 3). The EPaxos reduction at seven and nine replicas is partly an artefact of client timeouts rather than a property of the protocol’s scaling: each timed-out request occupies a client worker for the full client timeout, and the closed-loop accounting including that term matches every run at those sizes. The lower EPaxos

throughput at those sizes should therefore not be read as a scaling characteristic of the protocol.

Client concurrency produced a different pattern: EPaxos gained considerably more from high concurrency than Raft, and the gap was widest at the highest concurrency tested. This is consistent with the evaluated EPaxos implementation accepting proposals through several replicas while the evaluated Raft request path is centred on the elected leader, and a Raft-only follow-up confirmed the replica-scaling penalty persists at higher concurrency (43–51 % lower at nine replicas across c32–c256) with the leader’s CPU utilisation rising with concurrency while remaining concentrated on the leader. The experiment does not, however, isolate that protocol mechanism from implementation-level request processing, scheduling, and transport costs, and the leader approached but did not exceed its two-core allocation, so the measurements are consistent with a leader-centred write path without demonstrating a hard CPU capacity ceiling.

The latency results show the same condition dependence. Raft’s p50 latency rose monotonically with replica count while the EPaxos p50 stayed within a narrow band, and at the lowest concurrency tested the ordering of the two reversed. The measured latency relationship therefore depends on the operating condition rather than being a fixed property of either design.

The write-ratio and conflict-ratio sweeps produced no statistically significant throughput trend for the evaluated implementations, and the conflict-validation family shows that this is not an artefact of the workload failing to realise the configured fraction: the realised hot-key access pattern matched the configuration exactly, and three of the four implementations still showed no systematic response over the range in which the keypace retains cold keys. The fourth, nvb EPaxos, declined monotonically with the configured fraction, which makes conflict sensitivity implementation-specific rather than a property of the protocol family.

These results bear on prior comparisons. Howard and Mortier found implementation and configuration to matter more than the protocol family [3], which matches the spread we measure between implementations of the same family. van Renesse et al. and Tollman et al. reported that conflict changes the relative performance of the two designs and degrades the tail under geo-replication [7, 9]; here the configured fraction left the tail as well as the median unchanged for three of the four implementations (p99 ratios of 0.96–1.09 at 75 % against 0 %) and moved nvb EPaxos’s median and tail together. A mechanism that needs cross-site or same-key contention would not appear in this single-host hot-key workload, which is the configuration in which we can measure it.

Overall, RQ1 indicates that replica count and concurrency produced the most visible changes in the measured performance, while write ratio and conflict ratio produced weaker or implementation-specific effects. These findings characterise the evaluated implementations and workload rather than establishing that the corresponding factors are unimportant to the underlying consensus protocols.

5.2 RQ2: Failure and Communication Behaviour

RQ2 examines the behaviour of the evaluated implementations under replica failure and increasing communication cost. Failure of the Raft leader and failure of a non-leader replica have clearly different consequences in the tested configuration: leader termination interrupted writes until a replacement was elected, whereas terminating a Raft follower or an EPaxos replica produced interruptions about two orders of magnitude shorter (Table 4).

This is consistent with the role of the elected leader in the evaluated Raft implementation, but it should be read as a property of the tested failure mechanisms and implementations rather than as a universal distinction between leader-based and leaderless consensus. The election experiment shows that recovery relates to the number of disrupted elections, although the association with the total availability gap is partly mechanical because the imposed isolation interval contributes to that gap directly; once the isolation interval is removed, the residual association with the remaining recovery interval is weak and is treated as descriptive rather than causal.

The communication experiments reveal a second form of sensitivity. Under application-level injection the four implementations appeared to differ sharply, but applying the same nominal delay outside the implementations with `tc/netem` compressed the spread of retained throughput from roughly a factor of forty to a factor of two, and it also changed the ordering of the implementations. The extreme apparent sensitivity of the application-level condition therefore originates in how that mechanism interacts with each implementation’s send path rather than in the added delay as such: because it sleeps per message, an implementation that sends more messages per request, or that serialises its sends, is affected more strongly. The controlled condition is the evidence used here for delay sensitivity; the application-level measurements characterise each implementation’s send path. Communication cost affects latency as well as throughput under the controlled condition, but less divergently across implementations than the application-level condition suggested. The jitter sweep supports the same reading, with the qualification that the implemented jitter model raises the mean injected delay as well as its variance, so that experiment cannot separate the two.

Taken together, the failure and communication experiments show that adverse-condition behaviour depends both on the role of the failed replica and on the communication mechanism used to impose network cost, and that the communication penalty is of a similar order across the evaluated implementations once the same delay is introduced outside their internal message-processing paths.

5.3 RQ3: Implementation-Level Sources of Performance Difference

RQ3 asks how much of the measured performance difference between Raft and EPaxos is attributable to the concrete implementations rather than to the consensus algorithm families themselves. The results from the implementation, persistence, and communication experiments consistently indicate that the measured relationship depends substantially on implementation details.

The controlled communication-delay family is the primary basis for comparing implementation sensitivity to communication cost, and its baseline ordering does not correspond to the leader-based versus leaderless classification: the implementation with the highest zero-delay throughput is not the one that retains the most throughput once delay is applied (Section 4.8).

The persistence-matching family provides particularly direct evidence. The recorded baseline compares a durable HashiCorp Raft with an in-memory original EPaxos, and matching the storage mode reverses the ordering at both replica counts tested, every matched comparison being significant at $p < 0.001$ (Section 4.7, Table 6). The cost of persistence is itself implementation-specific rather than uniform, and the implementation that loses the largest fraction of its in-memory throughput when its log is persisted is the one that the recorded comparison placed against an in-memory EPaxos. The recorded baseline therefore combines consensus behaviour with a substantial difference in persistence configuration, and its EPaxos/Raft ratio cannot be treated as an intrinsic property of the two protocol families.

The resource measurements add a second observation. Mean per-replica CPU utilisation was similar between the two baseline implementations even though their throughput differs, and within the Raft scaling runs the elected leader consumed 2.2 to 3.8 times the CPU of a follower, the ratio growing with cluster size. The concentration that a leader-centred request path implies therefore does appear in resource use; what the throughput results do not show is a clean scaling penalty for it, because the EPaxos measurements at the largest sizes are confounded by client timeouts (Section 4).

The combined results therefore support an implementation-sensitive reading of the Raft–EPaxos comparison: measured throughput and latency arise from the interaction of consensus structure with persistence, request processing, scheduling, message handling, transport, workload, and failure conditions, so a difference between the evaluated implementations cannot be attributed to the algorithm family alone. This does not imply that algorithmic structure is irrelevant. The asymmetric failure behaviour, the different response to client concurrency, and the different latency scaling with replica count are all consistent with mechanisms arising from the respective consensus designs; the results indicate that those mechanisms are observed through concrete implementations, and that implementation costs can materially amplify, attenuate, or even reverse the measured relationship.

5.4 Implications for Interpreting the Comparison

The three research questions together show that a comparison based on a single implementation pair and a single local-network workload can conflate several distinct effects: the measured relationship changes with workload and system scale, with failure role and communication conditions, and with the persistence configuration, the message handling, and the mechanism used to introduce communication cost. The controlled network experiment demonstrates measurable differences in communication sensitivity without supporting a universal claim about the relative WAN performance of EPaxos and Raft, and the failure results show an availability cost for replacing the Raft leader without establishing that every leader-based system incurs the same cost: election timeout configuration, transport implementation, retry policy, and failure detection can all affect the measured recovery interval, and the same principle applies to the short recovery measured in the tested EPaxos replica-failure scenario.

For experimental comparisons, the results therefore indicate that persistence, transport, communication conditions, workload, failure configuration, and implementation-specific behaviour should be controlled or explicitly reported when attempting to separate protocol effects from system-level effects.

6 Threats to Validity

6.1 External Validity

All replicas run on a single host over a bridge network, so the measurements contain no wide-area latency, no real packet loss, and no multi-machine failure domains. The host is a WSL2 environment with 12 physical cores, placing a virtualisation layer between the containers and the hardware. Cluster sizes are 3–9, the keyspace is 1,000 keys, commands are uniform in size, and the measured phase is 20 s. The results therefore describe short, uniform, single-host runs rather than long-lived deployments with skewed access patterns, large payloads, or geographically distributed replicas.

The larger cluster sizes also introduce physical resource contention that cannot be completely separated from the replica-count effect. Nine replicas at 2.0 CPU requests each, together with the client and master, request 21 CPUs against a 12-core host, and each replica is allocated 2.0 CPUs while `GOMAXPROCS=4`, so Go scheduler parallelism exceeds the container CPU quota. Consequently, the 7- and 9-replica configurations should be interpreted as measurements under a resource-constrained local environment rather than as isolated evidence of protocol-level scaling. These configurations are retained because they represent the behaviour of the evaluated systems in this environment, but the 3- and 5-replica configurations provide a less resource-contended basis for the primary scaling comparison.

Reads are routed through consensus in both protocols and no Raft-specific read optimisation is enabled, so the read comparison is of equivalent consensus-facing paths rather

than each system’s fastest possible read path. Both systems are measured with identical CPU and memory allocations, which provides a controlled comparison but not a resource-tuned deployment.

Persistence is a source of implementation asymmetry. In the original baseline, the evaluated Raft implementations use persistent storage, while the EPaxos implementations use in-memory state. In particular, the nvb EPaxos implementation does not provide a built-in durable storage backend; no new persistence mechanism is implemented for this study. Consequently, the original latency and throughput measurements may include persistence costs that are specific to the evaluated implementations rather than to the consensus protocols themselves. A separate persistence-control experiment therefore evaluates memory and durable configurations where the respective implementations support both modes; its results are reported in Section 4.7 and show that the recorded ordering depends on which storage mode each implementation was run in. The nvb implementation is evaluated only in its supported in-memory configuration.

The EPaxos implementation is additionally modified to support the dependency-set sizes required by the 7- and 9-replica configurations. Results at those sizes therefore combine the protocol behaviour with the cost of the required implementation extension.

The controlled network-delay experiment applies fixed one-way delay to inter-replica traffic using operating-system-level network emulation rather than application-level sleeps. The delay is scoped to peer-to-peer replica traffic, leaving client-to-replica traffic unaffected. This approach avoids the adapter-level serialisation effects of application-level delay injection, but it still does not reproduce the full behaviour of a distributed physical network, including packet loss, congestion, multi-hop routing, or independent link failures. The earlier application-level delay and jitter measurements are retained as legacy experiments but are not treated as the primary evidence for clean network delay sensitivity (Section 4.8); jitter is not included in the controlled delay experiment.

The conflict ratio is specified through the workload’s key-access pattern, but the configured percentage does not by itself establish the amount of protocol-level contention observed by EPaxos. The conflict-validation experiment therefore records the realised hot-key access pattern together with available protocol-level execution indicators, including fast- and slow-path behaviour where exposed by the implementation (Section 4.9). Conclusions about conflict sensitivity are limited to effects supported by both the workload configuration and these observed execution characteristics.

The failure and election experiments use three replicas, so their recovery times characterise that configuration only. The failure scenarios are also not symmetric: the Raft leader-failure experiment removes the normal request endpoint, whereas the follower- and replica-failure experiments remove a non-central replica. The Raft leader-recovery interval depends on the configured heartbeat interval and election timeout, as well as on client timeout and reconnection behaviour. These parameters are therefore reported explicitly, and the

measured availability gap is interpreted as a result under the specified failure-detection configuration rather than as an implementation-independent recovery time for Raft.

The correctness harness verifies state convergence under the tested workloads and restart scenarios but does not constitute a formal verification of consensus correctness or establish general linearisability. In particular, the tested workloads do not exercise every possible pattern of conflicting operations and message reordering that could expose an ordering violation. The harness writes a distinct key per request, so it does not exercise concurrent writes to the same key, and no linearisability checker was run against the recorded histories; it is therefore a convergence check rather than an independent test of operation-order semantics under contention.

6.2 Statistical Conclusion Validity

Each configuration is measured ten times, and aggregates are reported as medians with non-parametric bootstrap 95% confidence intervals. Ten repetitions leave substantial uncertainty: measuring the same nominal configuration in four steady-state families produced medians that differed by 2–3% for both protocols, and the bootstrap intervals overlap in several comparisons whose medians differ, such as at seven and nine replicas. We therefore treat overlapping comparisons as inconclusive rather than as evidence of a small but established effect.

The suite contains many configurations and pairwise comparisons, and no multiplicity correction is applied (Section 3): a p -value quoted in the evaluation is the uncorrected value for that single comparison. Tests are reported next to the family they belong to, so the number of comparisons behind a claim is visible; we rely on comparisons that are well separated rather than on borderline significance. The separation is wide enough that the choice does not matter: every result reported as significant is at $p \leq 0.001$, and no family here contains more than ten tests, so a Holm–Bonferroni correction at $\alpha = 0.05$ would leave each of those results significant.

The election correlation analysis requires particular caution: the measured failed-election count is an outcome of the recovery process and is mechanically related to the injected isolation duration, and the 60 runs are grouped by six isolation targets (ten runs each). The observations should therefore not be treated as 60 fully independent samples. The correlation is interpreted primarily as a descriptive association rather than as independent evidence of a general causal relationship.

The 20s measurement window limits the characterisation of long-term steady-state behaviour, and the closed-loop client means the configured concurrency is a maximum number of in-flight requests rather than an independently controlled arrival rate. Outliers within clean runs are not removed; runs identified as affected by environmental interference are investigated separately and are not excluded solely because their performance is inconvenient for the comparison.

7 Conclusion

This study conducted a controlled experimental comparison of Raft and EPaxos as representative leader-based and leaderless consensus designs, varying replica count, client concurrency, workload composition, conflict ratio, failure mode, communication cost, and persistence configuration to distinguish effects associated with consensus structure from those introduced by implementation and experimental environment.

Increasing the replica count reduced throughput in both implementations, while the evaluated EPaxos configuration maintained higher throughput under high client concurrency; a Raft-specific scaling experiment showed this penalty persists as concurrency increases, with CPU utilisation increasingly concentrated at the leader. These observations are consistent with a leader-centred request path, although the experiments do not establish that protocol structure alone accounts for the observed differences. Workload composition produced no uniform response: only nvb EPaxos responded to the configured conflict fraction.

The failure experiments reveal another consequence of centralised request processing: in the tested three-replica configuration, leader failure interrupted write processing until a replacement leader was established, whereas follower failure in Raft and replica failure in EPaxos produced substantially shorter interruptions, characterising the observed failure behaviour rather than establishing protocol-independent recovery bounds.

The implementation-sensitivity experiments provide an important qualification to direct protocol-level comparisons. Implementations belonging to the same consensus family differed in absolute throughput and latency and did not necessarily respond identically to controlled communication delays; implementation-level mechanisms could materially alter the measured results without changing the underlying consensus algorithm. Absolute performance measurements should therefore be interpreted as properties of a particular implementation and environment, rather than as intrinsic performance constants of Raft or EPaxos.

Taken together, the findings do not support a universal performance ordering between leader-based and leaderless consensus: the measured relationship emerges from the interaction of these factors, and neither observation should be interpreted as a general superiority claim for either protocol family.

The study also illustrates a methodological consideration for empirical consensus research: controlled follow-up experiments, including persistence matching and communication-cost injection, provide useful evidence for determining whether an observed difference is robust to changes in the surrounding implementation environment, and performance is most informative when analysed as a function of controlled system conditions rather than reduced to a single throughput or latency figure.

Several limitations constrain the generalisation of these findings: the experiments were conducted in a single-host containerised environment with operating-system-level network emulation, a limited range of cluster sizes, and a closed-loop workload, and the correctness experiments verified convergence rather than linearisability.

Future work could extend the methodology to multi-host and geographically distributed environments, additional independent implementations, larger replica sets, and workloads with stronger contention and more diverse access patterns. Profiling at the storage, transport, scheduling, and application layers could further separate protocol-level behaviour from implementation overhead.

Acknowledgments

Generative AI tools were used during the preparation of this work for assistance with benchmark and laboratory code development, language editing, proofreading, and drafting and revision of manuscript text. The author independently defined the research questions, experimental design, overall methodology, and interpretation of the study. Experimental execution, data analysis, and verification of the reported results were conducted and checked by the author. AI-generated suggestions were reviewed and revised by the author, who is responsible for the final content and technical claims of the manuscript.

References

- [1] etcd Authors. 2025. etcd: Distributed Reliable Key-Value Store (raft package). <https://github.com/etcd-io/etcd/tree/main/raft>. Version v3.7.0.
- [2] HashiCorp. 2025. HashiCorp Raft: A Raft Consensus Protocol Implementation in Go. <https://github.com/hashicorp/raft>. Version v1.7.3.
- [3] Heidi Howard and Richard Mortier. 2020. Paxos vs Raft: Have We Reached Consensus on Distributed Consensus?. In *7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC 20)*. ACM, 1–9. doi:10.1145/3380787.3393681
- [4] Leslie Lamport. 1998. The Part-Time Parliament. *ACM Transactions on Computer Systems* 16, 2 (1998), 133–169. doi:10.1145/279227.279229
- [5] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There Is More Consensus in Egalitarian Parliaments. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP 13)*. ACM, Farmington, PA, 358–372. doi:10.1145/2517349.2522738
- [6] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*. USENIX Association, Philadelphia, PA, 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [7] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, 613–632. <https://www.usenix.org/conference/nsdi21/presentation/tollman>
- [8] Nathan van Benschoten. 2025. EPaxos: Egalitarian Paxos in Go. <https://github.com/nvanbenschoten/epaxos>. Commit 425bd36.
- [9] Robbert van Renesse, Nicolas Schiper, and Fred B. Schneider. 2015. Vive La Différence: Paxos vs. Leaderless Replication. In *Proceedings of the 28th ACM Symposium on Principles of Distributed Computing (PODC 15)*. ACM, 242–251. doi:10.1145/2767386.2767419